A COOKBOOK FOR BACKEND ENGINEERS

# Machine Learning Cookbook

Practical machine learning for engineers who ship — the workflow, the algorithms, the evaluation, and how to serve models in production.

Python · scikit-learn · pandas / NumPy · Regression · Classification · Evaluation · Deep learning · MLOps

---

Written for backend developers who now know Python and want practical ML — models that ship, not just math.

Every topic answers: what it is, how to build it, the pitfalls, and when NOT to use ML at all.

Edition 2026 · intuition over proofs · engineering over hype

# Table of Contents

# How to Use This Book

You're an engineer. You write code that follows rules *you* specify: `if this, then that`. Machine learning flips that — instead of writing the rules, you show a program lots of examples and it *learns the rules itself*. This book teaches you to do that practically: to frame a problem as ML, prepare data, train and evaluate models, and — the part most courses skip and you'll care about most — **serve them in production**.

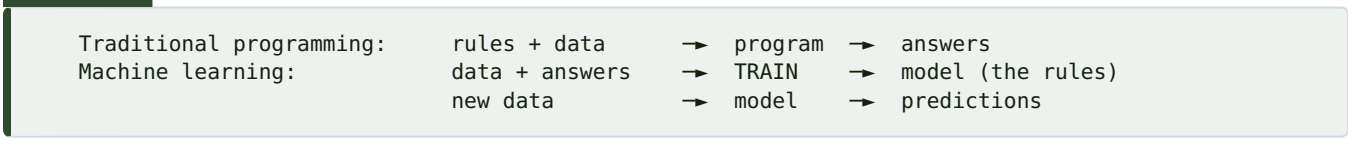
It's written for a backend developer who now knows Python (from the companion cookbooks) and wants ML that ships, not a math degree. So it favors **intuition over proofs**, uses `scikit-learn` and friends, and treats a model as what it really is to you: a component with inputs, outputs, a build pipeline, tests (evaluation), and a deployment. It connects directly to what you already know — the embeddings and LLMs from the RAG book are ML; FastAPI is how you'll serve a model.

Each chapter answers four questions:

1. **What is this and when do you use it?**
2. **How do you build it (in Python)?**
3. **What are the pitfalls?**
4. **When is ML the wrong tool?**

## The one idea: learn rules from examples

DIAGRAM




You give a learning algorithm **examples** (input features) paired with the **right answers** (labels), and it produces a **model** — a function that maps inputs to outputs. Then you feed the model *new* inputs and it predicts. The whole discipline is: get good examples, pick a model, train it, and rigorously check it generalizes to data it hasn't seen.

> **ENGINEER'S MENTAL MODEL**
>
> A model is a function you *fit* instead of *write*. Training is a build step (slow, offline, reproducible). Inference is a function call (fast, online — you'll wrap it in an API, Part 11). Evaluation is your test suite — except a model is never "correct," only "accurate enough on held-out data," which is why measurement (Part 8) is the heart of ML the way it's the heart of RAG. You already have the instincts; this book gives them ML vocabulary.

## The kinds of ML

| Type | You give it | It learns to | Example |
|---|---|---|---|
| **Supervised** | inputs + labels | predict labels for new inputs | spam/not-spam, price prediction |
| — regression | inputs + a **number** | predict a number | house price, demand forecast |
| — classification | inputs + a **category** | predict a category | fraud/legit, which of N classes |
| **Unsupervised** | inputs only | find structure | customer segments, anomalies |
| **Reinforcement** | an environment + rewards | act to maximize reward | game agents, robotics (niche here) |

Most practical ML — and most of this book — is **supervised**: you have labeled examples and want predictions. Deep learning (Part 10) is a powerful *technique* that spans these, not a separate category.

## When ML is the wrong tool

> **WHEN NOT TO USE MACHINE LEARNING**
>
> ML is for problems where the rules are too complex or fuzzy to write by hand *and* you have data with the answers. Don't use it when: **simple rules work** (a threshold, a lookup, a regex — don't train a model to check if a number is positive); you **don't have labeled data** (no examples to learn from — get data first, or it's not ML yet); you need **guarantees or explanations** the model can't give (safety-critical exact logic); or the problem is really **analytics** (a SQL `GROUP BY` answers it). ML trades determinism and simplicity for the ability to learn patterns — only make that trade when the pattern is the problem.

## What you'll be able to do

By the end: frame a business problem as an ML task, load and clean real data, engineer features, train regression and classification models, evaluate them honestly (the difference between a model that works and one that only *looks* like it does), reach for deep learning when it's warranted, and deploy a model behind an API with monitoring. Practical, shippable ML — grounded in the engineering you already do.

# Part 1 · The ML Workflow

ML beginners think the work is "pick an algorithm." Practitioners know the algorithm is a small, late step; the work is everything around it — framing, data, features, evaluation, and deployment. This part is the map every ML project follows, so the rest of the book has a place to hang.

## 1.1 The loop

DIAGRAM

```
1. FRAME    what are we predicting? is ML even right? what's "good enough"?
     │
2. DATA     collect, clean, explore, split (train / validation / test)
     │
3. FEATURES turn raw data into informative inputs (often the biggest lever)
     │
4. TRAIN    fit a model on the training set
     │
5. EVALUATE measure on held-out data; compare models; tune
     │        └──────────── iterate: back to features/train ──────────┐
     ▼                                                                 │
6. DEPLOY   serve the model behind an API; batch or online            │
     │                                                                 │
7. MONITOR  watch accuracy, latency, data drift ─── retrain ──────────┘
```

It's a **loop**, not a line: you'll cycle 3→5 many times, and monitoring in production (7) sends you back to retrain as the world changes. The single most important structural rule appears in step 2 and governs everything after it.

## 1.2 The golden rule: hold out a test set, and don't touch it

Split your data into **train** (fit the model), **validation** (tune choices), and **test** (a final, untouched estimate of real-world performance). The test set is sacred: you look at it **once**, at the end. If you tune your model against the test set, you're teaching it to the answers and your "accuracy" is a lie that collapses in production.

PYTHON

```python
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3,
random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5,
random_state=42)
# 70% train, 15% validation, 15% test
```

**COMMON MISTAKE**

**Evaluating on data the model trained on.** A model can memorize its training data and score 99% on it while being useless on anything new — this is *overfitting* (Part 8), and testing on training data hides it completely. The number that matters is performance on data the model has **never seen**. Every credible ML result is a held-out number; a training-set score is marketing, not measurement.

## 1.3 Frame before you fit

Before any code, answer:

- **What exactly are we predicting?** A number (regression) or a category (classification)? For whom, and to drive what decision?
- **What does "good enough" mean?** The metric and the bar (Part 8). "95% accuracy" is meaningless without context — for a fraud detector where 1% of transactions are fraud, a model that predicts "never fraud" is 99% accurate and totally worthless.
- **What's the baseline?** The dumb approach (predict the average; predict the most common class; a simple rule). Your fancy model must beat it, or it isn't earning its complexity.
- **Do we have the data?** Labeled examples, enough of them, representative of production. No data, no ML.

**PRODUCTION TIP**

Always build the **dumbest baseline first** — predict the mean, the majority class, or a hand-written rule — and measure it. It takes ten minutes, sets the bar every model must clear, and surprisingly often is *good enough* to ship, saving you a modeling project entirely. A model that beats a strong baseline by a little may not be worth the complexity it adds to your system.

## 1.4 Where the effort really goes

| Stage | Share of effort (typical) | Where quality comes from |
|---|---|---|
| Framing | small but decisive | picking the right problem + metric |
| Data + cleaning | **the majority** | most real ML time is here |
| Features | large | often the biggest accuracy lever |
| Model choice + training | **small** | a few lines; rarely the bottleneck |
| Evaluation | ongoing | the thing that keeps you honest |
| Deploy + monitor | large in production | where models earn value or rot |

Internalize this: **data and features beat algorithms.** A simple model on great features and clean data beats a fancy model on poor ones almost every time — the same lesson as "retrieval beats the LLM" from the RAG book.

**EXERCISE 1.1**

Pick a concrete prediction problem from your own work (e.g. "will this user churn?", "how long will this job take?"). Write down: what you're predicting, regression or classification, the metric and the bar for "good enough", the dumb baseline, and whether you actually have labeled data. If you can't fill all five in, you've found the real first task — usually getting data.

# Part 2 · The Math You Actually Need

You do not need a math degree to do practical ML — libraries handle the calculus. But a handful of *intuitions* make everything else make sense, and their absence is why ML feels like magic. This part gives you those intuitions, kept light and practical. Skim it now; refer back when a later chapter uses a term.

## 2.1 Data as vectors and matrices

Everything becomes numbers. One example (a house, a user, an email) is a **vector** of feature values: `[1500, 3, 2, 1995]` (sqft, beds, baths, year). Your whole dataset is a **matrix** — rows are examples, columns are features. The labels are a vector `y`. That's the shape of every ML dataset:

DIAGRAM

```
        feature1  feature2  feature3        label
ex1 [    1500       3         2      ]  →   320000
ex2 [     900       2         1      ]  →   210000     X (matrix)   y (vector)
ex3 [    2100       4         3      ]  →   450000
```

A model is a function `f(X) → y`: it maps a feature vector to a prediction. "The embedding vectors" from the RAG book are this same idea — data as points in numeric space.

## 2.2 A model is a function with parameters

Take the simplest model, a line: `prediction = w · feature + b`. The `w` (weight) and `b` (bias) are **parameters** — the numbers the model learns. Training means *finding the parameter values that make the predictions best match the labels*. A complex model just has many more parameters (a neural network has millions), but the idea is identical: dials the training process turns to fit the data.

## 2.3 Loss: measuring wrongness

To "make predictions match labels," you need to *measure* how wrong the model is. That's the **loss function** — a single number, higher = worse. For regression, a common one is mean squared error (average of `(prediction − actual)²`). Training is the search for parameters that **minimize the loss**.

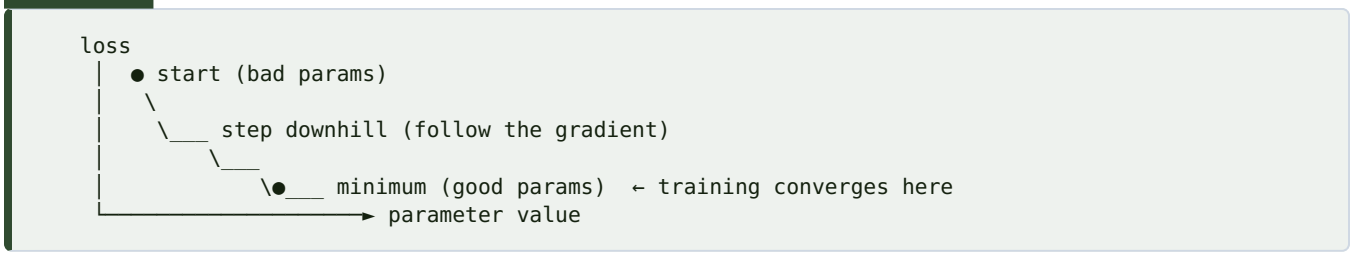
PYTHON

```python
# the essence of a loss
def mse(predictions, actuals):
    return sum((p - a) ** 2 for p, a in zip(predictions, actuals)) / len(actuals)
```

## 2.4 Gradient descent: how learning happens

How does the model find loss-minimizing parameters? **Gradient descent**: start with random parameters, compute which direction reduces the loss (the *gradient*, i.e. the slope), take a small step that way, repeat. It's rolling downhill on the loss surface until you reach a low point.

DIAGRAM




The **learning rate** is the step size: too big and you overshoot and bounce around; too small and training crawls. You'll meet it as a knob in most models. This one idea — *nudge the parameters in the direction that lowers the loss* — powers everything from linear regression to giant neural networks (Part 10).

**ENGINEER'S MENTAL MODEL**

Gradient descent is an optimization loop: `while not good_enough: params -= learning_rate * slope_of_loss`. It's iterative refinement toward a goal, like tuning a config by measuring, adjusting, and re-measuring — automated. You won't implement it (libraries do), but knowing the loss goes *down* by *steps* whose *size* is the learning rate demystifies "training" entirely.

## 2.5 The three statistics terms you'll actually use

- **Distribution** — how values are spread (mean, spread/variance, skew). You look at distributions to understand and clean data (Part 4).
- **Correlation** — how two variables move together. High correlation between a feature and the label is promising; between two features can mean redundancy.
- **Probability** — classifiers usually output a probability ("0.87 fraud"), and you choose a threshold to turn it into a decision (Part 7). Understanding that a prediction is often a *confidence*, not a certainty, matters for how you use it.

**PRODUCTION TIP**

Don't rabbit-hole into math before building. You can train, evaluate, and ship useful models with exactly the intuitions above — a model is a parameterized function, training minimizes a loss via gradient descent, and you judge it on held-out data. Deepen the math *when a specific problem demands it* (a custom loss, a weird optimizer), not preemptively. Momentum from building beats theory you'll forget.

**EXERCISE 2.1**

By hand or in a few lines of Python: fit a line to five points `(x, y)` by trying a couple of slope/intercept values, computing MSE for each, and picking the lower — you've just done what gradient descent automates. Then plot the five points and your best line. This tiny exercise makes "parameters, loss, minimize" concrete before any library hides it.

# Part 3 · The Toolkit & Setup

Python owns machine learning because of its libraries. A small, stable core does almost everything in classic ML; you already know how to manage Python environments from the Python cookbook. This part gets you set up and introduces the tools you'll use in every later chapter.

## 3.1 The core stack

| Library | Role | Think of it as |
|---|---|---|
| **NumPy** | fast numeric arrays/matrices | the array math everything sits on |
| **pandas** | tabular data (DataFrames) | SQL/Excel in Python; load, clean, explore |
| **scikit-learn** | classic ML: models, metrics, pipelines | the workhorse — one consistent API for everything |
| **matplotlib** / seaborn | plotting | seeing your data and results |
| **Jupyter** | interactive notebooks | a REPL you can save — the ML dev environment |
| (later) PyTorch / TensorFlow | deep learning | neural nets (Part 10) |

For classic ML — which is most practical ML — **scikit-learn + pandas + NumPy** is the whole kit.

## 3.2 Setup

SHELL

```
uv init ml-lab && cd ml-lab
# from the Python cookbook: uv = your toolchain
uv add numpy pandas scikit-learn matplotlib jupyter
uv run jupyter lab                                  # opens the notebook UI
```

Notebooks are where ML work happens: you load data, poke at it, plot, train, and evaluate cell by cell, keeping state between steps. It's an interactive loop that suits ML's "explore, try, measure" rhythm far better than a script.

**ENGINEER'S MENTAL MODEL**

A Jupyter notebook is a persistent REPL with saved output — like a scratch file where each block runs independently and keeps variables in memory. Use notebooks for **exploration and training** (the messy, iterative part), then move the finished pipeline into proper `.py` modules for production and serving (Part 11). Notebooks are the lab; your codebase is the factory. Don't ship the notebook.

## 3.3 The scikit-learn API you'll use everywhere

scikit-learn's genius is one consistent interface for every model: `.fit()` to train, `.predict()` to use, `.score()` to check. Learn it once, use it for linear regression, random forests, k-means — everything.

PYTHON

```python
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()      # 1. create (with hyperparameters)
model.fit(X_train, y_train)           # 2. train on features + labels
preds = model.predict(X_test)         # 3. predict on new features
score = model.score(X_test, y_test)   # 4. quick built-in metric
```

Swap `RandomForestClassifier` for any other estimator and the four lines are identical. This uniformity is why you can try five models in an afternoon.

## 3.4 A complete tiny example

The whole workflow in a dozen lines, using a built-in dataset:

PYTHON

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)                                  # features, labels
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_tr, y_tr)                                               # train
preds = model.predict(X_te)                                         # predict on held-out
print(accuracy_score(y_te, preds))                                 # honest score
```

That's a real, trained, evaluated classifier. Everything else in this book makes each line better: better data (Part 4), better features (Part 5), the right model (6–7), honest evaluation (8), and shipping it (11).

**PRODUCTION TIP**

Set a `random_state` (seed) everywhere there's randomness — splits, model init — so your results are **reproducible**. "It scored 0.94 yesterday and 0.89 today" with no code change is almost always an unseeded split, and it makes debugging and comparing models impossible. Reproducibility is a first-class engineering concern in ML, exactly as in the rest of your stack.

**EXERCISE 3.1**

Set up the environment and run the twelve-line Iris example in a notebook. Then, changing only the model line, swap in `LogisticRegression` and `KNeighborsClassifier` (same `.fit/.predict` API) and compare accuracies. You've just experienced the scikit-learn uniformity that makes model comparison cheap — the workflow you'll repeat all book.

# Part 4 · Data: Loading, Cleaning & Exploring

Most of ML is data work, and most model failures are data failures. This part is the unglamorous, decisive craft of getting data into shape and *understanding it* before you model. Skipping it is the number-one reason ML projects produce impressive-looking, useless models.

## 4.1 Loading and inspecting with pandas

pandas DataFrames are your SQL-table-in-Python. Load, then *look*:

PYTHON

```python
import pandas as pd
df = pd.read_csv("data.csv")          # or read_sql, read_parquet, read_json

df.head()                             # first rows — eyeball the shape
df.info()                             # columns, types, null counts
df.describe()                         # min/max/mean/quartiles per numeric column
df["target"].value_counts()           # class balance (crucial for classification)
df.isnull().sum()                     # missing values per column
```

`df.describe()` and `df.info()` are the first things you run on any dataset — they reveal ranges, types, and missingness at a glance.

## 4.2 Cleaning: the real work

Real data is messy. The common fixes:

| Problem | Typical handling |
|---|---|
| **Missing values** | drop rows/columns, or *impute* (fill with mean/median/mode, or a model) |
| **Wrong types** | parse dates, cast strings to numbers |
| **Duplicates** | `df.drop_duplicates()` |
| **Outliers** | investigate (bug or real?), cap, or leave — don't blindly delete |
| **Inconsistent categories** | normalize ("USA"/"U.S."/"us" → one value) |
| **Wrong scale/units** | fix before modeling |

PYTHON

```
df = df.drop_duplicates()
df["age"] = df["age"].fillna(df["age"].median())     # impute missing ages
df["signup"] = pd.to_datetime(df["signup"])          # fix a type
```

## 4.3 Exploratory Data Analysis (EDA)

Before modeling, *understand* the data: distributions, relationships, and surprises. Plot histograms of each feature, a correlation heatmap, and the target against key features. EDA is where you catch the problems that would otherwise silently wreck your model, and where feature ideas (Part 5) come from.

PYTHON

```python
import matplotlib.pyplot as plt
df.hist(figsize=(12, 8)); plt.show()            # distributions of every numeric column
df.corr(numeric_only=True)                       # correlations (feature↔target, feature↔feature)
df.plot.scatter(x="sqft", y="price")            # does the relationship look real?
```

## 4.4 Data leakage: the silent killer

**Leakage** is when information sneaks into training that won't be available at prediction time (or that encodes the answer). It produces models that score brilliantly in testing and fail in production — the most dangerous ML bug because it *looks like success*.

Classic leaks:

- A feature that's a **proxy for the label** (using `was_refunded` to predict `will_churn` when refunds happen *because* of churn).
- **Future information** (using a value recorded *after* the event you're predicting).
- **Fitting preprocessing on all the data** before splitting — the scaler/imputer "sees" the test set. Always split *first*, then fit transforms on **train only** (Part 5).
- **Duplicate or near-duplicate rows** split across train and test.

**COMMON MISTAKE**

A model with suspiciously high accuracy (0.99 on a hard problem) is almost always **leakage**, not brilliance. Before celebrating, ask: "could any feature contain information I wouldn't have at prediction time?" Check which features the model relies on most (Part 8) — if the top one is a sneaky proxy for the answer, you've found the leak. Distrust results that are too good.

**PRODUCTION TIP**

Split into train/test **before you clean or transform anything**, then apply every fitted transformation (imputation values, scalers, encoders) learned from *train* to the test set — never the reverse. scikit-learn `Pipeline` (Part 5) enforces this automatically and is the single best guard against leakage. "Fit on train, apply to test" is the discipline that keeps your evaluation honest.

**WHEN NOT TO TRUST YOUR DATA**

If the data was collected differently from how production data will arrive (different population, time period, or process), a model trained on it won't generalize no matter how good your pipeline — *distribution shift*. And if labels are noisy or wrong (mislabeled examples), the model learns the noise. Sometimes the right move is to fix data collection first, not to model harder. No algorithm rescues fundamentally unrepresentative or mislabeled data.

**EXERCISE 4.1**

Take any real CSV (a Kaggle dataset or your own). Run `info` / `describe` / `value_counts`, handle missing values and duplicates, and make three plots: a histogram grid, a correlation heatmap, and the target vs. one feature. Write down two things the EDA revealed that you didn't expect — and check whether any feature might leak the answer.

# Part 5 · Feature Engineering

Features are the inputs your model actually sees, and turning raw data into good features is frequently the biggest lever on accuracy — bigger than the choice of algorithm. This part covers the transformations you'll apply to almost every dataset, and the pipeline discipline that keeps them leak-free.

## 5.1 What a feature is

A **feature** is one input column. Feature engineering is creating and shaping those columns so the model can find the pattern: extracting signal from raw data (a date → `day_of_week`, `is_weekend`, `days_since`), combining columns (price / sqft → price_per_sqft), and encoding non-numeric data into numbers the model can use. Domain knowledge shines here — you know which derived quantities matter.

## 5.2 The transformations you'll always need

**Scaling** — many models care about the *magnitude* of features, so a feature ranging 0–1,000,000 (income) drowns out one ranging 0–1 (a ratio). Standardize so features are comparable.

PYTHON

```python
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)      # fit on TRAIN only
X_test_scaled = scaler.transform(X_test)            # apply to test (no re-fit!)
```

**Encoding categoricals** — models need numbers, so categories (`"red"`, `"blue"`) must be converted:

| Encoding | How | Use for |
|---|---|---|
| **One-hot** | one 0/1 column per category | nominal categories (color, country) |
| **Ordinal** | map to ordered ints | ordered categories (small<med<large) |
| **Target/mean** | replace with the label's mean for that category | high-cardinality (careful: leakage-prone) |

PYTHON

```python
import pandas as pd
X = pd.get_dummies(df, columns=["color", "country"])   # one-hot encoding
```

**Handling missing values as a step** (imputation) and **creating features** (binning a number into ranges, extracting parts of a date/string) round out the usual kit.

| Which models care about scaling? | |
|---|---|
| **Do** care (scale them) | linear/logistic regression, SVM, k-NN, k-means, neural nets |
| **Don't** care | tree-based models (decision trees, random forests, gradient boosting) |

## 5.3 Pipelines: bundle transforms with the model

The professional way to apply transformations is a scikit-learn `Pipeline`, which chains preprocessing + model into one object. It fits every step on train and applies them on test *automatically* — eliminating the leakage mistake from Part 4 and giving you one object to train, evaluate, save, and serve (Part 11).

PYTHON

```python
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import GradientBoostingClassifier

numeric = ["age", "income", "sqft"]
categorical = ["country", "plan"]

pre = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                      ("scale", StandardScaler())]), numeric),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical),
])

model = Pipeline([("pre", pre), ("clf", GradientBoostingClassifier())])
model.fit(X_train, y_train)          # ALL preprocessing fit on train, then model
model.predict(X_test)                # same transforms applied automatically
```

**PRODUCTION TIP**

Always wrap preprocessing and the model in a single `Pipeline`. It's not just tidy — it's correctness (no leakage, transforms fit only on train) *and* deployability: you save one object and, at serving time, raw input goes in and a prediction comes out with identical preprocessing to training. A model saved without its preprocessing is a production bug waiting to happen — the input the API receives won't match what the model was trained on.

**COMMON MISTAKE**

Fitting a scaler or encoder on the *whole* dataset before splitting, or forgetting to apply the *same* transform at prediction time. Both silently corrupt results: the first leaks test information into training; the second feeds the model differently-shaped data in production than in training, tanking real-world accuracy while your offline tests looked fine. Pipelines prevent both — use them by default.

**WHEN NOT TO OVER-ENGINEER FEATURES**

Don't manufacture hundreds of features hoping something sticks — it invites overfitting (Part 8), leakage, and noise, and it slows everything down. Start with the obvious, informative features, get a baseline, and add features only when EDA or error analysis suggests a specific signal you're missing. With tree-based models especially, a few strong features beat a swamp of weak ones. Feature *quality* over feature *quantity*.

**EXERCISE 5.1**

Take a dataset with mixed numeric + categorical columns and some missing values. Build a single `Pipeline` with a `ColumnTransformer` that imputes and scales the numerics, one-hot-encodes the categoricals, and trains a model — then `fit` on train and `predict` on test in two lines. Confirm you never touched the test set during preprocessing. Add one engineered feature (e.g. a ratio) and see if the score improves.

# Part 6 · Supervised Learning I — Regression

Regression predicts a **number**: a price, a duration, a demand, a score. It's the gentlest place to meet supervised learning because the model (a weighted sum of features) and its metrics are intuitive. This part covers building regressors and judging them honestly.

## 6.1 Linear regression: the foundation

Linear regression fits the best straight-line relationship: `prediction = w₁·f₁ + w₂·f₂ + … + b`. Training finds the weights that minimize the error (Part 2). It's fast, interpretable (each weight tells you a feature's effect), and a strong baseline.

PYTHON

```python
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
preds = model.predict(X_test)
print(model.coef_, model.intercept_)     # the learned weights + bias (interpretable)
```

## 6.2 Regression metrics: how wrong, in what units

| Metric | Meaning | Notes |
|---|---|---|
| **MAE** (mean absolute error) | average absolute miss, in target units | intuitive: "off by $23k on average" |
| **RMSE** (root mean squared error) | like MAE but punishes big misses more | sensitive to outliers |
| **R²** (coefficient of determination) | fraction of variance explained (1 = perfect, 0 = no better than predicting the mean) | unitless; compare models |

PYTHON

```python
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score
print("MAE ", mean_absolute_error(y_test, preds))
print("RMSE", root_mean_squared_error(y_test, preds))
print("R2  ", r2_score(y_test, preds))
```

Pick the metric that matches the cost of errors: MAE if all misses hurt equally, RMSE if big misses are disproportionately bad. Always compare against the dumb baseline (predict the mean → $R^2$ of 0).

## 6.3 Beyond linear

When the relationship isn't a straight line, stronger regressors capture curves and interactions without you specifying them:

| Model | Strength | Watch out |
|---|---|---|
| Linear / Ridge / Lasso | fast, interpretable; Ridge/Lasso *regularize* to fight overfitting | assumes roughly linear |
| Decision tree | captures non-linearity; no scaling needed | overfits alone |
| **Random forest** | many trees averaged — robust, strong default | less interpretable |
| **Gradient boosting** (XGBoost/ LightGBM) | often best-in-class on tabular data | more tuning |

**Regularization** (Ridge = L2, Lasso = L1) is worth knowing: it penalizes large weights to keep the model simpler and less prone to overfitting — Lasso can even zero out useless features (automatic feature selection).

**ENGINEER'S MENTAL MODEL**

For tabular data — rows and columns, the data you have in databases — **gradient-boosted trees** (XGBoost/LightGBM) are the pragmatic default that usually wins, and deep learning usually *doesn't* help. Reach for neural nets (Part 10) for images, audio, and text; reach for boosted trees for the business tables you actually work with. Knowing this saves you from over-reaching for deep learning on problems trees solve better and cheaper.

**COMMON MISTAKE**

Judging a regressor by $R^2$ alone, or on the training set. A high training $R^2$ with a much lower test $R^2$ is overfitting (Part 8). And $R^2$ can look decent while the model is useless for your decision — always translate error into the real units (MAE/RMSE) and ask "is being off by this much acceptable for what this prediction drives?" A metric is only meaningful against the cost of being wrong.

**PRODUCTION TIP**

Start with linear regression as your baseline (fast, interpretable — its weights tell you which features matter and how), then try a random forest and gradient boosting and compare on held-out data. Often the boosted model wins by enough to justify its complexity; sometimes the linear model is within a hair and you ship *that* for its speed and interpretability. Let the measured gap, not the hype, decide.

**EXERCISE 6.1**

On a regression dataset (house prices, bike rentals), train linear regression, a random forest, and gradient boosting. Report MAE, RMSE, and $R^2$ on the test set for each, plus the dumb baseline (predict the mean). Look at the linear model's coefficients: which features push the prediction up or down, and does it match your intuition? Decide which model you'd ship and why.

# Part 7 · Supervised Learning II — Classification

Classification predicts a **category**: spam/not-spam, fraud/legit, which-of-N. It's the most common ML task in products, and its metrics are subtler than regression's — "accuracy" is often the *wrong* number and will lie to you. This part covers building classifiers and, crucially, measuring them correctly.

## 7.1 The models

Same scikit-learn API, different estimators:

PYTHON

```python
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
preds = model.predict(X_test)                 # the class
probs = model.predict_proba(X_test)[:, 1]     # the probability of the positive class
```

| Model | Strength |
|---|---|
| **Logistic regression** | fast, interpretable baseline; outputs probabilities |
| Decision tree | interpretable, non-linear; overfits alone |
| **Random forest** | robust strong default |
| **Gradient boosting** (XGBoost/LightGBM) | usually best on tabular data |
| k-NN, SVM, Naive Bayes | situational classics |

Note `predict_proba`: most classifiers output a **probability** (0.87), and you apply a **threshold** (default 0.5) to decide the class. Moving that threshold trades false positives against false negatives — a product decision, not a default.

## 7.2 Why accuracy lies: the imbalance trap

**Accuracy** = fraction correct. It's useless when classes are imbalanced. Fraud is ~0.1% of transactions, so a model that predicts "legit" for *everything* is 99.9% accurate and catches zero fraud. You must look deeper.

## 7.3 The confusion matrix and its metrics

Every classification result is four counts:

DIAGRAM

```
                    predicted POSITIVE   predicted NEGATIVE
actual POSITIVE       True Positive         False Negative   (missed it)
actual NEGATIVE       False Positive        True Negative
                      (false alarm)
```

From these come the metrics that actually matter:

| Metric | Formula (intuition) | Answers |
|---|---|---|
| **Precision** | TP / (TP + FP) | of what I flagged, how much was right? (false-alarm cost) |
| **Recall** (sensitivity) | TP / (TP + FN) | of what's actually positive, how much did I catch? (miss cost) |
| **F1** | harmonic mean of precision & recall | one number balancing both |
| **ROC-AUC** | ranking quality across all thresholds | how well it separates classes |

PYTHON

```python
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
print(confusion_matrix(y_test, preds))
print(classification_report(y_test, preds))       # precision, recall, F1 per class
print(roc_auc_score(y_test, probs))
```

**Precision vs. recall is a business trade-off.** A cancer screen wants high **recall** (never miss a case, tolerate false alarms). A spam filter wants high **precision** (never junk a real email, tolerate some spam getting through). You tune the threshold to the cost of each error type — there is no universally right answer.

## 7.4 Handling imbalance

When one class is rare: use **class weights** (`class_weight="balanced"` — tell the model to care more about the rare class), **resample** (oversample the minority, e.g. SMOTE; or undersample the majority), and **judge by precision/recall/F1/AUC, never accuracy**. Pick the threshold from a precision-recall curve, not the 0.5 default.

**COMMON MISTAKE**

Reporting **accuracy on an imbalanced problem** and declaring victory. It's the classic ML self-deception: 99% accurate and completely broken. For any problem where the classes aren't roughly balanced — which is most interesting problems — lead with the confusion matrix, precision, recall, and F1 (or AUC), and state which error type is costlier and why. Accuracy is the metric that hides your model's failure.

**PRODUCTION TIP**

Decide **which error hurts more** before you model — false positives or false negatives — because it determines your metric, your threshold, and how you handle imbalance. Then set the decision threshold deliberately from a precision-recall curve to hit the trade-off your product needs, rather than accepting 0.5. And expose the *probability* to downstream systems where you can (e.g. "87% likely fraud → send to review" vs a hard block) — a confidence is more useful than a bare yes/no.

**WHEN NOT TO TRUST A CLASSIFIER'S CONFIDENCE**

A model's `predict_proba` is a *calibrated-ish* score, not a true probability unless you've checked (or applied calibration). Don't treat "0.9" as "90% of these are truly positive" without verifying calibration, especially for high-stakes decisions. And a classifier confidently classifies inputs unlike anything it trained on — it doesn't know what it doesn't know. Guard critical decisions with thresholds, human review, and out-of-distribution checks.

**EXERCISE 7.1**

On an imbalanced dataset (fraud, churn, or make one imbalanced), train a classifier and print accuracy, the confusion matrix, precision, recall, F1, and ROC-AUC. Note how accuracy flatters a bad model. Then add `class_weight="balanced"`, move the threshold, and watch precision and recall trade off — pick the operating point you'd actually ship for a fraud reviewer's workload.

# Part 8 · Evaluation, Overfitting & Tuning

This is the chapter that separates people who *do* ML from people who *think* they do. A model that scores well on the data it trained on tells you nothing; the entire skill is estimating how it'll perform on data it has never seen, and tuning it without fooling yourself. Get this right and everything else is technique.

## 8.1 Overfitting and underfitting

Every model sits somewhere on this spectrum:

DIAGRAM

```
UNDERFIT ──────────── JUST RIGHT ──────────── OVERFIT
too simple            captures the pattern    memorized the training data
bad on train & test   good on train & test    great on train, BAD on test
(high bias)           (the goal)              (high variance)
```

- **Underfitting:** the model is too simple to capture the pattern — poor on both training and test data. Fix: a more powerful model, better features.
- **Overfitting:** the model memorized the training data, including its noise — great on training, poor on test. Fix: more data, a simpler model, regularization, fewer features.

You *diagnose* by comparing training vs. test scores: a big gap (great train, poor test) = overfitting; both poor = underfitting.

## 8.2 Cross-validation: a trustworthy estimate

A single train/test split is one noisy estimate — you might get lucky or unlucky with which rows landed where. **k-fold cross-validation** splits the data into k parts, trains k times (each time holding out a different part as the test), and averages. You get a more reliable performance estimate *and* a sense of its variance.

PYTHON

```python
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring="f1")   # 5-fold
print(scores.mean(), "±", scores.std())     # average performance and how stable it is
```

## 8.3 Hyperparameter tuning

Models have **hyperparameters** — settings *you* choose, not learned (tree depth, number of trees, regularization strength, learning rate). Tuning searches for good values, evaluated by cross-validation on the **validation** data (never the test set).

PYTHON

```python
from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(
    GradientBoostingClassifier(),
    {"n_estimators": [100, 300], "max_depth": [2, 3, 4], "learning_rate": [0.05,
0.1]},
    cv=5, scoring="f1",
)
grid.fit(X_train, y_train)
print(grid.best_params_, grid.best_score_)   # best combo by cross-validated score
best = grid.best_estimator_
```

`GridSearchCV` tries every combination; `RandomizedSearchCV` samples randomly (better when the space is large). Tune on train/validation, then report the final number **once** on the untouched test set.

## 8.4 The one rule that governs all of it

**The test set is touched exactly once, at the very end.** Every time you look at test performance and change something in response, you leak the test set into your decisions and your final number becomes optimistic. Tune on validation (or cross-validation within the training data); reserve the test set for a single, final, honest estimate you report and don't act on.

**COMMON MISTAKE**

"Tuning to the test set" — running the model on the test data, tweaking hyperparameters or features, re-running, and repeating until the test score looks great. You've now overfit *to the test set itself*, and production will be worse than your number promised. Use a separate validation set (or cross-validation) for all tuning; the test set is a one-shot final exam, not a practice sheet.

**PRODUCTION TIP**

Report performance with **cross-validation mean ± std**, not a single split — the standard deviation tells you whether a model that "beat" another is genuinely better or within noise. Before shipping, also do **error analysis**: look at the specific examples the model gets wrong, grouped by feature. Real insight (a whole segment failing, a leaky feature, a data-quality issue) comes from reading errors, not from staring at a single aggregate metric.

**WHEN NOT TO KEEP TUNING**

Chasing another 0.5% on your metric is usually the wrong use of time once you've cleared the bar. Marginal tuning gains are fragile (they may be noise) and rarely matter as much as better data, a fixed leak, or actually shipping and monitoring (Part 11). Stop tuning when you beat the baseline by a meaningful, stable margin and the errors you have left are acceptable — then ship and learn from production.

**EXERCISE 8.1**

Take a model and deliberately overfit it (e.g. a very deep tree), then compare its training score to its cross-validated test score — see the gap. Now regularize (shallower tree / add `min_samples_leaf`) and watch the gap close. Run `GridSearchCV` over a small hyperparameter grid with 5-fold CV, then report the winner's performance *once* on a held-out test set. That single disciplined number is your real result.

# Part 9 · Unsupervised Learning

Sometimes you have data but no labels — no "right answers" to predict. Unsupervised learning finds *structure* in the data itself: natural groupings, lower-dimensional representations, anomalies. It's less about prediction and more about discovery, and a few techniques cover most practical needs.

## 9.1 Clustering: finding natural groups

**Clustering** groups similar examples without being told the groups — customer segmentation, grouping documents, finding communities. **k-means** is the workhorse: you pick `k` (the number of clusters) and it partitions the data into `k` groups by similarity.

PYTHON

```python
from sklearn.cluster import KMeans
km = KMeans(n_clusters=4, random_state=42, n_init="auto")
labels = km.fit_predict(X_scaled)     # each row gets a cluster id 0..3
# inspect: what characterizes each cluster? (group means per feature)
```

You choose `k` with methods like the "elbow" (plot within-cluster error vs. `k` and look for the bend) or silhouette score, plus domain sense. Scale your features first (Part 5) — k-means uses distances. Other clustering algorithms (DBSCAN, hierarchical) handle non-spherical clusters or unknown `k`.

## 9.2 Dimensionality reduction: fewer, richer features

High-dimensional data (many features) is slow, hard to visualize, and prone to overfitting. **PCA (Principal Component Analysis)** compresses many correlated features into a few "components" that capture most of the variation — useful for visualization (squash to 2-D and plot), speeding up models, and denoising.

PYTHON

```python
from sklearn.decomposition import PCA
pca = PCA(n_components=2)              # squash to 2 dimensions for plotting
X_2d = pca.fit_transform(X_scaled)
print(pca.explained_variance_ratio_)  # how much information each component keeps
```

For *visualizing* clusters or embeddings, **t-SNE** and **UMAP** are popular non-linear alternatives (great for 2-D plots, but for viewing only — don't feed their output into models the way you would PCA).

**ENGINEER'S MENTAL MODEL**

You've already met unsupervised learning without the label: the **embeddings** in the RAG book are learned vector representations, and *vector search* is essentially finding nearest neighbors in that space — cousins of clustering and dimensionality reduction. Anomaly detection (flag the points far from any cluster) is how you'd spot fraud or outages without labeled examples. The "find structure in vectors" intuition transfers directly.

## 9.3 Where it's genuinely useful

| Task | Technique | Example |
|---|---|---|
| Segment customers/users | clustering (k-means) | marketing personas, cohorts |
| Anomaly / outlier detection | clustering / isolation forest | fraud, defects, outages |
| Visualize high-dim data | PCA / UMAP / t-SNE | see structure in embeddings |
| Speed up / denoise features | PCA | reduce 200 features to 20 |
| Topic discovery | clustering on embeddings | group documents by theme |

## 9.4 The honest caveat

Unsupervised results are **harder to validate** — there's no held-out accuracy, because there's no ground truth. "Are these the right clusters?" is partly a judgment call, guided by metrics (silhouette) but ultimately by whether the groups are *useful and interpretable* for your goal. This makes unsupervised learning more exploratory and more subjective than supervised.

**WHEN NOT TO REACH FOR UNSUPERVISED LEARNING**

Don't cluster when you actually have labels and a prediction goal — that's a supervised problem, and using the labels will beat guessing structure. Don't treat clusters as ground truth; they're hypotheses to validate, not facts. And don't apply PCA reflexively — if you only have ten features, reducing them rarely helps and costs interpretability. Reach for unsupervised methods when you genuinely lack labels and want to *understand* or *segment* data, not as a default step.

**COMMON MISTAKE**

Running k-means on unscaled features (so one large-range feature dominates the distance and defines the clusters by itself), or picking `k` arbitrarily and reading meaning into whatever falls out. Scale first, choose `k` with the elbow/silhouette *and* domain sense, and always **inspect** the clusters (what distinguishes each group?) before trusting them — an uninterpreted cluster is a number, not an insight.

**EXERCISE 9.1**

Take an unlabeled dataset (or drop the labels from a labeled one). Scale it, run k-means for k = 2…8, plot the elbow, pick a `k`, and characterize each cluster by its feature means — give each a human name. Then PCA the data to 2-D and scatter-plot it colored by cluster to *see* whether the groups are real and separated.

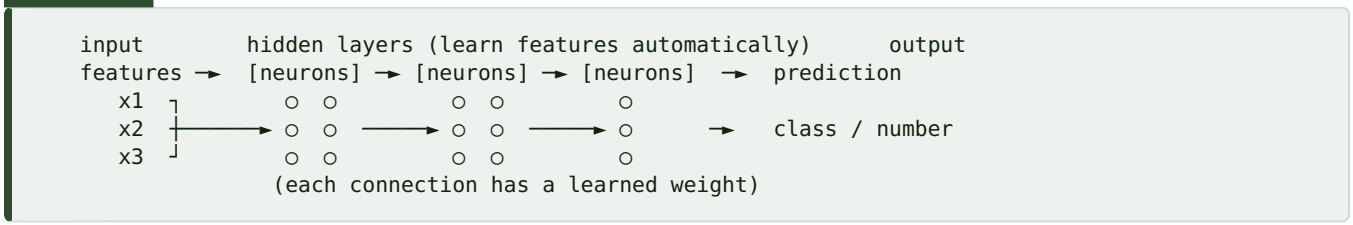
# Part 10 · A Taste of Deep Learning

Deep learning powers the headline AI — image recognition, speech, and the LLMs behind the RAG book. This part demystifies it: what a neural network is, when it beats classic ML (and when it doesn't), and how it connects to what you already know. It's a taste, not a deep dive — enough to know when to reach for it and how to start.

## 10.1 From a line to a network

Recall Part 2: a simple model is `w·x + b`. A **neuron** is exactly that, followed by a non-linear "activation" function. Stack neurons into **layers**, and layers into a **network**, and you get a function that can approximate extremely complex patterns — because each layer transforms the previous layer's output, learning progressively richer representations.

DIAGRAM




Training is still **gradient descent** (Part 2) minimizing a loss — the algorithm that computes the gradients through all those layers is called **backpropagation**. Same core idea as linear regression, just millions of parameters and many layers.

## 10.2 The key difference: learned features

Classic ML needs *you* to engineer features (Part 5). Deep learning's superpower is **learning features automatically** from raw data — pixels, audio samples, characters — discovering the useful representations itself. That's why it dominates images, audio, and language, where hand-engineering features is hopeless. It's also why it needs **lots of data and compute**: it's learning the features *and* the task.

## 10.3 When to use deep learning — and when not

| Use deep learning for | Use classic ML (trees) for |
|---|---|
| Images, video | **tabular/structured data** (rows & columns) |
| Audio, speech | most business prediction problems |
| Text / language (LLMs) | small-to-medium datasets |
| Very large datasets | when you need interpretability |

| Use deep learning for | Use classic ML (trees) for |
|---|---|
| Unstructured, high-dimensional raw input | when compute/latency is tight |

**ENGINEER'S MENTAL MODEL**

The RAG cookbook was applied deep learning: **embeddings** come from neural networks, and **LLMs** are enormous ones. You've already been *using* deep learning models via APIs — this chapter is what's inside them. For most of your own tabular problems, though, gradient-boosted trees (Parts 6–7) still win; don't reach for a neural net on a spreadsheet because it sounds advanced. Deep learning is a specialized power tool, not the default.

## 10.4 The frameworks and transfer learning

**PyTorch** (research-favorite, dominant) and **TensorFlow/Keras** are the two frameworks. A tiny taste with Keras:

PYTHON

```python
from tensorflow import keras
model = keras.Sequential([
    keras.layers.Dense(64, activation="relu", input_shape=(n_features,)),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(1, activation="sigmoid"),     # binary classification
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.fit(X_train, y_train, epochs=10, validation_split=0.2)
```

The most practical deep-learning technique for engineers is **transfer learning**: take a big model someone else trained on massive data (an image model, a language model) and adapt it to your task with a little of your own data — or just *use* it via an API (embeddings, an LLM). You rarely train big networks from scratch; you stand on pretrained ones. That's exactly what RAG does with LLMs and embeddings.

**WHEN NOT TO USE DEEP LEARNING**

Don't reach for neural nets on tabular/structured data with a few thousand rows — boosted trees are simpler, faster, more interpretable, and usually more accurate there. Don't use it when you lack the data (deep nets are data-hungry and overfit small sets), the compute, or the interpretability your use case requires. And don't train from scratch what you can get from a pretrained model or API. Deep learning shines on unstructured data at scale — match it to that, not to everything.

**PRODUCTION TIP**

For 90% of the tabular ML you'll do as a backend engineer, **you won't need deep learning at all** — scikit-learn and gradient boosting cover it. When you do need it (images, text, audio), start with a **pretrained model** via transfer learning or a hosted API rather than training your own; it's faster, cheaper, and better. Save custom deep-learning training for when a pretrained model genuinely can't do the job.

**EXERCISE 10.1**

Take the same tabular dataset you used for classification (Part 7) and train both a gradient-boosted tree *and* a small Keras neural net on it. Compare accuracy, training time, and effort. Notice the tree likely wins with far less fuss — the lesson that deep learning isn't the default for tabular data. Separately, use a pretrained embedding model (from the RAG book) on some text and observe deep-learned features you didn't engineer.

# Part 11 · MLOps — Shipping & Serving Models

A trained model in a notebook has zero value; a model serving predictions in production has all of it. This is the chapter where your backend skills matter most — serving, versioning, monitoring — and where most ML projects quietly die because the team could train a model but never operationalized it. This is your home turf.

## 11.1 Save the whole pipeline, not just the model

Persist the entire fitted `Pipeline` (preprocessing + model, Part 5) so serving applies identical transformations to training. Save the training data schema and the library versions alongside it.

PYTHON

```python
import joblib
joblib.dump(pipeline, "model_v3.joblib")     # the WHOLE pipeline (preprocess + model)
# later, in the serving process:
pipeline = joblib.load("model_v3.joblib")
pred = pipeline.predict(raw_input_dataframe)  # raw input in, prediction out
```

**COMMON MISTAKE**

Saving only the model and re-implementing preprocessing by hand in the serving code. The two drift apart — a scaler mean, an encoding, a column order differs — and the model silently receives inputs unlike its training data, degrading predictions with no error. **Save the fitted pipeline as one artifact** so "raw input → prediction" is one call with guaranteed-consistent preprocessing.

## 11.2 Serving: a prediction API

Online serving is a normal API — wrap the pipeline in a FastAPI endpoint (Python cookbook). Load the model **once** at startup, validate input with Pydantic, predict per request.

PYTHON

```python
from fastapi import FastAPI
from pydantic import BaseModel
import joblib, pandas as pd

app = FastAPI()
```

```python
pipeline = joblib.load("model_v3.joblib")     # load ONCE at startup, not per request

class Applicant(BaseModel):
    age: int
    income: float
    country: str
    plan: str

@app.post("/predict")
def predict(a: Applicant):
    X = pd.DataFrame([a.model_dump()])         # one-row frame with training columns
    proba = float(pipeline.predict_proba(X)[0, 1])
    return {"risk_score": proba, "decision": "review" if proba > 0.7 else "approve",
            "model_version": "v3"}
```

| Serving mode | When | How |
|---|---|---|
| **Online / real-time** | per-request predictions (fraud check at checkout) | API endpoint (above) |
| **Batch** | score many rows on a schedule (nightly churn scores) | a job/queue (Queues cookbook) writing to a table |
| **Streaming** | react to events | consumer + model |

**ENGINEER'S MENTAL MODEL**

Model serving is just an API with an unusual dependency. Everything from the companion cookbooks applies verbatim: load the model once (not per request), validate input (Pydantic), set **timeouts**, add **caching** for repeated inputs (Redis), run heavy/batch scoring on a **queue** (BullMQ/Celery), and put it in a **container** (Docker). A `/predict` endpoint is a `/query` endpoint whose backend is a `.joblib` file instead of a database. You already know how to run this.

## 11.3 Versioning and reproducibility

Treat models like releases: version the **model artifact**, the **data** it was trained on, the **code**, and the **metrics** it achieved. Return the model version in every prediction response (as above) so you can trace a decision back to the exact model that made it. Tools like MLflow or DVC formalize this (experiment tracking, model registry, data versioning); a disciplined folder + git-LFS + a metrics file is a fine start.

## 11.4 Monitoring and drift — models rot

Unlike normal code, a model **degrades over time even if you never touch it**, because the world changes and production data drifts away from what it trained on (**data/concept drift**). A fraud model trained on last year's patterns slowly gets worse as fraud evolves. So monitoring is not optional:

| Watch | Why |
|---|---|
| Prediction **latency** & errors | it's still an API |
| Input **distribution** vs. training | data drift — inputs no longer look like training |
| **Prediction** distribution | sudden shifts signal a problem |
| **Accuracy** on labeled outcomes (when labels arrive) | the real signal — is it still right? |
| Business metric it drives | the ultimate test |

When drift or decay shows up, you **retrain** on fresh data (the loop closes, Part 1). Automate this pipeline where the stakes justify it.

**PRODUCTION TIP**

Log every prediction with its inputs, output, model version, and (later, when it arrives) the true outcome — this is what lets you *measure production accuracy*, detect drift, debug bad decisions, and build the next training set from reality. It's the exact "log the query, chunks, and answer" discipline from the RAG book. Also: ship the **baseline or a simple model first**, behind the same serving/monitoring rig — get the pipeline live, then improve the model within it. A deployed simple model beats a brilliant notebook model.

**WHEN NOT TO BUILD HEAVY MLOPS**

Don't stand up a full MLflow/Kubeflow/feature-store platform for one model scored nightly — a `joblib` file, a FastAPI endpoint (or a cron batch job), version tags, and prediction logging is plenty, and far less to operate. Scale the MLOps machinery to the number of models and the stakes: a single internal model needs a fraction of what a fleet of customer-facing models does. Match the platform to the problem, as with every other tool in this series.

**EXERCISE 11.1**

Take a trained pipeline, `joblib.dump` it, and build a FastAPI `/predict` endpoint that loads it once and validates input with Pydantic — returning a prediction plus the model version. Containerize it (Docker cookbook). Add prediction logging (input + output + version). Then simulate drift by feeding it inputs shifted from the training distribution and describe what a drift monitor would flag. You've now shipped and operated a model.

# Part 12 · Pitfalls, Ethics & Cheat Sheet

The whole book condensed: how to choose an algorithm, the traps that sink real projects, the responsibility that comes with prediction, and a one-page reference.

## 12.1 Choosing an algorithm

DIAGRAM

```
What are you predicting?
  │
  ├─ a NUMBER ──────────► Regression (Part 6): start linear → gradient boosting
  │
  ├─ a CATEGORY ────────► Classification (Part 7): start logistic → gradient boosting
  │                        (imbalanced? weight/resample, judge by precision/recall)
  │
  ├─ GROUPS / structure, no labels ─► Clustering / PCA (Part 9)
  │
  └─ images / audio / text ─► Deep learning or a pretrained model / API (Part 10)

Tabular data? → gradient-boosted trees (XGBoost/LightGBM) are the default winner.
Small data or need interpretability? → linear/logistic or a shallow tree.
```

## 12.2 The pitfalls that sink projects

| Pitfall | Symptom | Fix |
|---|---|---|
| **Data leakage** | too-good scores; fails in prod | split first; "fit on train only"; audit top features (P4/5) |
| **Overfitting** | great on train, poor on test | simpler model, regularize, more data, CV (P8) |
| **Testing on training data** | inflated, meaningless scores | held-out test set, touched once (P1/8) |
| **Accuracy on imbalanced data** | 99% and useless | precision/recall/F1/AUC (P7) |
| **No baseline** | can't tell if the model helps | predict mean/majority first (P1) |
| **Preprocessing not saved** | prod ≠ training inputs | save the whole Pipeline (P5/11) |
| **Ignoring drift** | model quietly rots | monitor + retrain (P11) |
| **Wrong metric** | optimizes the wrong thing | pick the metric from the decision's costs (P7/8) |
| **Bad/unrepresentative data** | model learns noise | fix data, not the model (P4) |

**The meta-rule:** when results look wrong, suspect the **data and the evaluation** before the algorithm. That's where the bugs live.

## 12.3 Responsible ML

Models make decisions about people, and they can cause real harm:

- **Bias & fairness:** a model trained on biased historical data *learns and amplifies* that bias (hiring, lending, policing). Check performance across subgroups, not just in aggregate — a model can be accurate overall and discriminatory for a group.
- **Explainability:** high-stakes decisions (credit, medical, legal) often *require* an explanation. Prefer interpretable models there, or add explanation tools (feature importance, SHAP). "The model said so" is not an acceptable reason to deny someone a loan.
- **Privacy:** be deliberate about training on personal data; models can leak it.
- **Feedback loops:** a model that influences the world it predicts (recommend → people click → training data → recommend) can spiral. Watch for it.

> **WHEN NOT TO DEPLOY A MODEL**
>
> Don't ship a model whose errors you can't live with, can't explain where an explanation is owed, or that performs unequally across groups in a way that causes harm — *even if the aggregate metric is great*. And don't automate a high-stakes decision end-to-end when a human-in-the-loop (model suggests, human decides) is safer. The ability to predict is not the same as the right to decide automatically.

## 12.4 The workflow, one page

| Step | Do | Key idea |
|---|---|---|
| Frame (P1) | define target, metric, baseline; is ML right? | pick the right problem |
| Data (P4) | load, clean, EDA, split FIRST | most of the work; watch leakage |
| Features (P5) | scale, encode, engineer, **Pipeline** | fit on train only |
| Model (P6/7) | regression or classification; start simple → boosting | trees win on tabular |
| Evaluate (P8) | held-out + cross-val; tune on validation | test set once |
| Deploy (P11) | save pipeline, FastAPI/batch, version | it's an API |
| Monitor (P11) | latency, drift, accuracy; retrain | models rot |

## 12.5 The glossary

| Term | Meaning |
|---|---|
| Feature / label | input column / the answer to predict |
| Parameter / hyperparameter | learned by training / set by you |

| Term | Meaning |
|---|---|
| Loss | how wrong the model is (minimized in training) |
| Gradient descent | the optimization that minimizes loss |
| Overfit / underfit | memorized training data / too simple |
| Bias / variance | error from being too simple / too sensitive |
| Cross-validation | rotate held-out folds for a stable estimate |
| Precision / recall | of flagged, how right / of actual, how many caught |
| Regularization | penalize complexity to fight overfitting |
| Drift | production data diverging from training data |
| Transfer learning | adapt/use a pretrained model on your task |

**YOU'RE READY**

Machine learning is not sorcery — it's a disciplined engineering loop: frame the problem, get clean representative data, engineer honest features, train a fittingly-simple model, **evaluate without fooling yourself**, and serve and monitor it like the production component it is. Your backend instincts — pipelines, testing, versioning, deployment, monitoring — are exactly what make ML *ship*. Start with a baseline on real data, measure ruthlessly, and add complexity only when the numbers earn it. That's the whole craft.